VR-SPARSEQUANT: ACCURATE FP4 QUANTIZATION WITH 2:4 SPARSITY ON VERA RUBIN

Samir Khaki [1] **Wei-Chen Wang** [1] **Song Han** [2] **Ryan Hanrui Wang** [1]

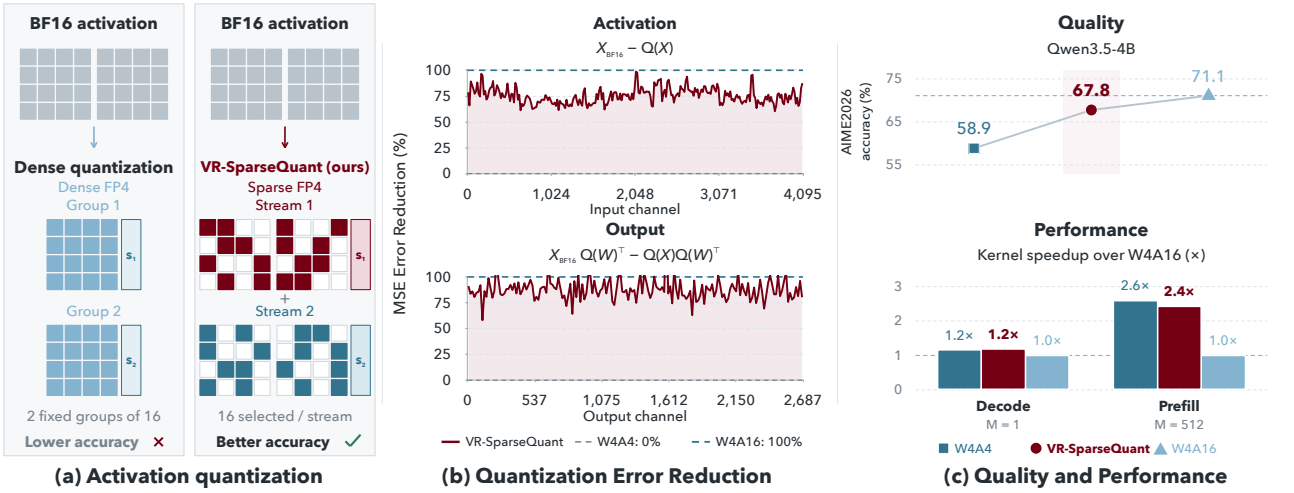


Figure 1. **Structured sparsity for accurate and efficient FP4 inference. (a)** Two 2:4 sparse FP4 streams approximate a dense activation block while retaining dense FP4's value budget. Overlapping positions receive a second contribution that reduces their reconstruction error. **(b)** Plots show the reduction in activation and layer-output mean squared error (MSE) achieved by VR-SparseQuant. **(c)** VR-SparseQuant improves Qwen3.5-4B AIME2026 accuracy over W4A4; Rubin GEMMs retain similar speed to dense FP4 and outperform W4A16.

ABSTRACT

In recent years, GPUs have incorporated increasingly specialized support for low-precision computation, from Hopper's FP8 to Blackwell's FP4 Tensor Cores. Precision formats have evolved alongside this hardware, from MXFP4 and NVFP4 on Blackwell to E2M1 FP4 with direct UE5M3 block scaling on Vera Rubin, which we term as **VRFP4**. These advances help serve larger language models at lower cost, but four-bit activation quantization can substantially degrade model quality. We introduce VR-SparseQuant, which uses Vera Rubin's elementwise 2:4 sparsity to recover accuracy while retaining native FP4 execution. By allowing two sparse streams to overlap, we use the same FP4 value budget as dense quantization to refine activations with larger errors. Offline scale-ratio calibration avoids runtime search, while a shared-weight GEMM accumulates both streams efficiently. On Nemotron-3-Nano-30B-A3B, VR-SparseQuant reduces WikiText-2 perplexity from **6.85** to **5.57** versus dense RTN W4A4. Including activation preparation, our native Rubin kernels for VRFP4 achieve up to $\mathbf{3.1}\times$ GEMM and $\mathbf{2.7}\times$ MoE speedup over BF16. To the best of our knowledge, this is the first study of adaptive activation quantization using Vera Rubin's elementwise 2:4 structured sparse FP4 instructions.

1 INTRODUCTION

Recent advances in large language model (LLM) inference have made low-bit execution practical for interactive generation and large-batch serving (Lin et al., 2024; 2025). These workloads occupy different points on the latency–throughput curve, with bottlenecks that change as weights are reused. At small decode batches, reading weights can dominate linear layers; larger batches and prompt processing reuse them across more tokens, making matrix-multiplication throughput increasingly important. Low-bit inference must therefore reduce both weight traffic and arithmetic cost to serve these operating regimes.

Weight-only quantization lowers memory footprint and traf-

[1]Nebius Applied AI, San Francisco, CA, USA [2]Massachusetts Institute of Technology, Cambridge, MA, USA. Correspondence to: Ryan Hanrui Wang <wang@nebius.com>.

fic while retaining higher-precision activations (Lin et al., 2024). Native FP4 also lowers compute cost, but requires four-bit activations encoded online, unlike weights quantized and packed offline. This encoding must preserve quality with little work and fit native instructions. Preparing models for NVIDIA's Vera Rubin platform therefore requires designing quantization around its arithmetic, scales, and sparse layouts (NVIDIA, 2026i;g).

Dense FP4 gives every activation one four-bit code and a shared block scale. Fixed slots simplify online packing but allocate values uniformly despite unequal errors. When large values set the scale, smaller values may be poorly resolved; smoothing and adaptive scales improve the fit (Xiao et al., 2023; Cook et al., 2025). Some positions still benefit more from a second contribution correcting rounding error than others. Existing methods add residual channels or sparse–dense branches (Meng et al., 2026a;b; Mak et al., 2026). Reallocating a fixed value budget instead requires choosing which positions to cover and correct while constructing native operands online.

Structured sparsity makes these choices through local position metadata while keeping value count and operand format fixed. Blackwell's compact sparse FP4 path selects adjacent pairs; Vera Rubin selects two individual positions per quartet (NVIDIA, 2025c; 2026g). Neither variable-length records nor input-dependent weight permutations are needed.

We introduce VR-SparseQuant (VR-SQ), which represents an activation as the sum of two structured 2:4 sparse matrices with different scales to better separate value ranges and improve quantization accuracy. We refer to each matrix as a *stream*, which retains the input's logical shape but stores only two FP4 values per quartet, together with scales and position metadata. Together, they match dense FP4's value count and nonzero arithmetic, but their sparsity patterns can overlap: the fine stream can represent an uncovered value or correct the coarse stream's approximation. Overlap leaves another position uncovered, so it is useful when correction removes more error than the omission introduces (Figure 1a).

Scales and positions are coupled, so a joint search repeatedly encodes each block. After smoothing and weight quantization, we instead calibrate the scale ratio offline. It ties the fine scale to the coarse scale while both track each block's range. Two local selections then assign coarse values and fine residual corrections without searching scale pairs online.

A preparation kernel constructs both streams together. A single sparse GEMM shares staged weight tiles across both streams and uses one FP32 accumulator, avoiding duplicate global-memory weight-tile loads, materialized intermediate outputs, and a combining pass. The kernels apply Rubin's UE5M3 block scales directly to E2M1 values, a format we call **VRFP4**.

We evaluate dense and mixture-of-experts language models and vision-language models on language modeling, reasoning, agentic tasks, and visual understanding. GPT-OSS (OpenAI, 2025) and MiniMax (MiniMax, 2026) test adaptation from released MXFP4 and NVFP4 checkpoints without original BF16 weights. Coverage studies progressively quantize more projection families; ablations examine smoothing, scales, and assignment. Including online activation encoding, native GEMM and MoE on Vera Rubin achieve 1.7–$3.1\times$ and 1.4–$2.7\times$ speedup over BF16, respectively; dense FP4 comparisons quantify the additional latency of this allocation.

To the best of our knowledge, this is the first study to design and evaluate adaptive activation quantization using Vera Rubin's compact elementwise sparse FP4 instructions. It connects a representation that improves model quality with kernels that execute it natively on Rubin. The broader contribution is a practical use of structured sparsity to allocate precision: sparse position metadata lets each input determine where a fixed budget of FP4 values is spent.

2 Preliminaries: FP4 on Vera Rubin

For a linear layer $Y = XW^\top$ with M rows, K input channels, and N output channels, the representation depends on which positions an operand may store and which values share a scale.

2.1 Elementwise Sparse FP4

A 2:4 sparse operand stores two values and their positions per quartet along K. Sparse Tensor Cores multiply only these entries, halving nonzero arithmetic relative to a dense product.

The allowed positions depend on the instruction format (Figure 2). Ampere FP16 and Hopper FP8 support elementwise 2:4 sparsity. Blackwell's compact sparse FP4 path selects two adjacent pairs from eight positions; Rubin's elementwise FP4 path selects two individual positions per group of four (NVIDIA, 2025c; 2026g). Both retain half the values, but neither mask family contains the other.

Our selected Rubin path shares one scale across 16 stored values (32 logical positions) and uses G32 weights.

2.2 Block Scales and Group Size

Microscaling combines low-bit values with scales shared across small blocks (Darvish Rouhani et al., 2023). E2M1 uses one sign bit, two exponent bits, and one fraction bit. Group size G governs scale sharing; the scale format determines range and precision.

NVFP4 combines unsigned E4M3 (UE4M3) scales for 16-value blocks with an FP32 tensor scale (NVIDIA, 2026f): $q_j = Q_4(t_j/(g_T s_b))$ and $\hat{t}_j = g_T s_b q_j$, where Q_4 rounds to E2M1, s_b is the block scale, and g_T is the tensor scale. Ru-

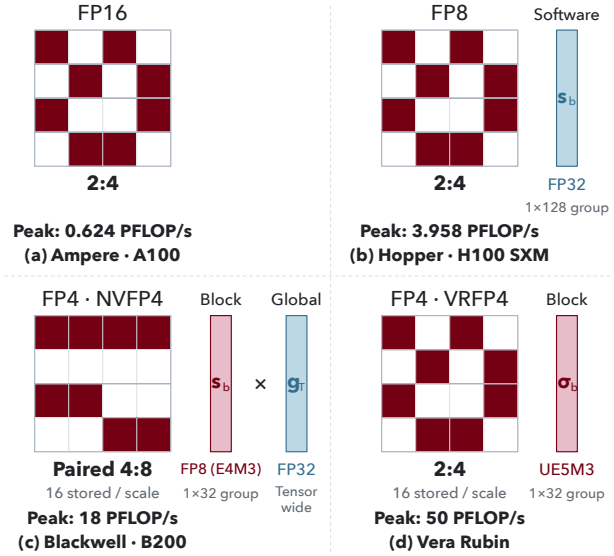


Figure 2. **Rubin brings elementwise 2:4 sparsity to the compact FP4 path.** Grids show 16 logical positions along K. Blackwell's compact FP4 path selects adjacent pairs; Rubin's elementwise path selects individual values within each quartet. Bars show scaling: the Hopper example uses software group scales, Blackwell uses FP8 block and FP32 global scales, and our Rubin path uses direct UE5M3 scales (NVIDIA, 2025c; 2026g; NVIDIA). Throughput labels are manufacturer per-GPU sparse peaks, not measured VR-SparseQuant performance (NVIDIA, 2026b;d;c;h).

bin's UE5M3 retains three fraction bits while widening the exponent range (NVIDIA, 2026g). We use it for direct block scaling, $\hat{t}_j = \sigma_b q_j$, and call the resulting representation VRFP4. The matrix instruction applies σ_b directly, eliminating the tensor-level $g_X g_W$ epilogue multiplier. Group size is specified separately.

Comparing scaling schemes at fixed group size. Table 1 compares VRFP4's direct UE5M3 scaling with NVFP4's bi-level microscaling on Qwen3.5-4B-Base (Qwen Team, 2026b). Within each G16 or G32 pair, only the scaling scheme changes. NMSE measures reconstruction error in the weights, WikiText-2 activations, and resulting outputs of one MLP down-projection. The two schemes yield nearly identical NMSE and perplexity in both comparisons, showing that direct block scaling preserves the quality of bi-level microscaling.

3 VR-SparseQuant

VR-SparseQuant allocates four FP4 values per quartet through two sparse streams; position metadata adapts this allocation to each input within a fixed operand layout. Figure 3 summarizes the workflow: the block maximum and a calibrated ratio set two grids, two greedy selections assign values, and one shared-weight GEMM accumulates both products.

Table 1. **VRFP4 closely matches NVFP4's reconstruction error.** Each G16 or G32 pair compares the scaling schemes at fixed group size under unsmoothed RTN. NMSE uses the weights, sampled activations, and outputs of one Qwen3.5-4B-Base MLP down-projection; PPL covers all 248 projections.

Format	G	NMSE↓			PPL↓
		Weight	Activation	Output	WikiText-2
BF16	—	0	0	0	8.05
NVFP4	16	8.95e−3	8.20e−3	1.51e−2	8.66
VRFP4	16	8.96e−3	8.21e−3	1.51e−2	8.67
NVFP4	32	1.03e−2	1.13e−2	1.90e−2	8.85
VRFP4	32	1.03e−2	1.13e−2	1.90e−2	8.86

3.1 Two-Stream Quantization

We encode the prepared input $U = \mathrm{BF16}(X \odot b)$, where b is a positive channel transform fixed during model preparation. Its inverse is absorbed into the weights (Section 3.4). For a block $u \in \mathbb{R}^{32}$ of this input, we write $u \approx a_1 + a_2$, where each stream stores two FP4 values per consecutive group of four and one scale for the block. The scales S_1 and S_2 define coarse and fine grids. Across the streams, a logical position can receive zero, one, or two selected contributions, while each stream retains its fixed 2:4 layout. A stream's *sparsity pattern* specifies its selected positions; a selected code can itself be zero. Section 4 accounts for the additional metadata and scales.

If the two sparsity patterns are disjoint, every activation is represented once; we call this assignment *complementary*. Allowing overlap lets the fine stream encode the coarse stream's residual. This trades coverage for correction: an overlap is useful when the error removed at one position exceeds the error introduced by leaving another position uncovered.

Figure 4 shows these choices in Qwen3.5-4B-Base activations. Squares and triangles on the axes have one nonzero contribution; off-axis circles have two. The fine contribution can be positive or negative, adjusting the coarse reconstruction in either direction. Combining the grids thus makes additional reconstruction levels available without increasing the FP4 value budget.

Encoding each block requires two choices: the scales of the two grids and the positions selected by each stream. Jointly searching both online would repeat assignment for each candidate scale pair. To keep encoding lightweight, we calibrate the scale ratio offline and fix it at inference. We then derive both scales from the block maximum and assign values through two greedy selections.

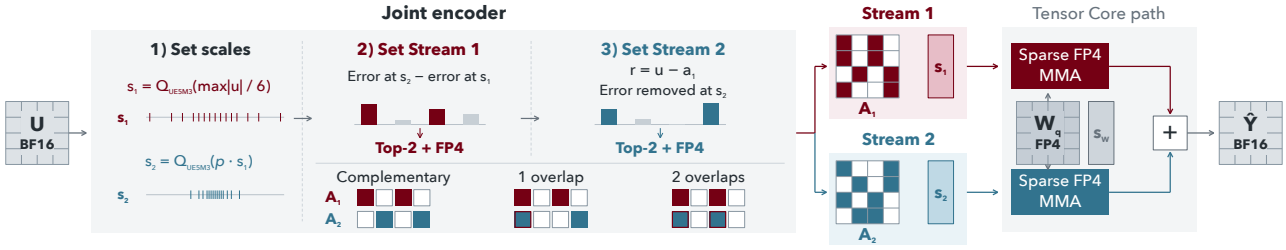


Figure 3. **Online allocation within a fixed sparse format.** The block maximum and a calibrated ratio set two FP4 scales. Greedy selections allocate coarse values and fine residual corrections within two fixed-size 2:4 streams. One sparse GEMM reuses quantized weights and accumulates both products in FP32 before converting the output to BF16.

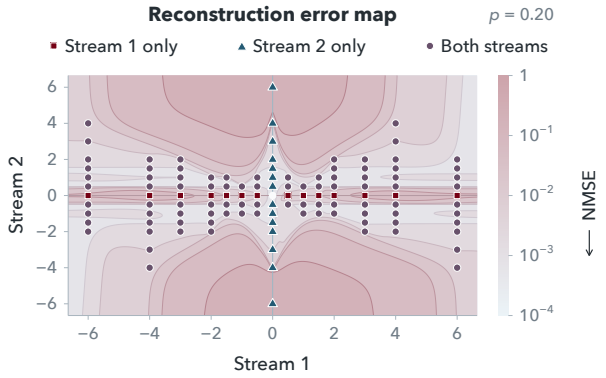


Figure 4. **Two streams provide finer reconstruction levels.** Observed pairs $(a_1/S_1, a_2/S_2)$ from Qwen3.5-4B-Base at $p = 0.20$. Squares and triangles show a contribution from stream 1 or 2 alone; circles show both. Shading interpolates each pair's mean normalized squared residual, $((u - a_1 - a_2)/S_1)^2$, on a log scale. Different pairs summarize different inputs.

3.2 Scale Selection

The coarse scale follows the current block maximum, while the calibrated ratio p determines the fine scale:

$$S_1 = \mathrm{RN}_{\mathrm{UE5M3}}(\max|u|/6),$$
$$S_2 = \mathrm{RN}_{\mathrm{UE5M3}}(pS_1), \qquad 0 < p \le 1. \tag{1}$$

Here 6 is the largest E2M1 magnitude. For finite, nonnegative inputs, $\mathrm{RN}_{\mathrm{UE5M3}}$ applies round-to-nearest, ties-to-even conversion with finite saturation, clamps the scale code to $[1, 254]$, and returns its decoded value. This encoder policy keeps both scales positive, including for all-zero blocks. Both scales apply to the same 32 positions and adapt to each block, while p stays fixed.

Reducing p gives the fine grid smaller reconstruction steps near zero but narrows its range. Calibration minimizes activation-induced output error (Section 3.5), leaving no runtime scale search.

3.3 Greedy Stream Assignment

With the scales fixed, the remaining online decision is which positions fill each stream's two slots. To avoid searching pairs of sparsity patterns, we first choose the best complementary assignment, then fix its coarse stream and select fine contributions from the residual. Both decisions reduce to selecting two gains within each quartet.

Let

$$Q(v, S) = S\ \mathrm{RN}_{\mathrm{E2M1}}(v/S) \tag{2}$$

denote quantization followed by dequantization. We compute $q_1 = Q(u, S_1)$ and $q_2 = Q(u, S_2)$ for the block. Assigning position i to the coarse grid instead of the fine grid reduces squared error by

$$g_{1,i} = (u_i - q_{2,i})^2 - (u_i - q_{1,i})^2. \tag{3}$$

Retaining the two largest-gain entries of q_1 per quartet and zeroing the others forms a_1. Since complementary error changes only through these gains, this selection minimizes it at the given scales.

We then quantize the residual $r = u - a_1$. At positions selected by the first stream, r contains quantization error; elsewhere, it contains the original value. For $c = Q(r, S_2)$, the benefit of retaining a fine-stream contribution is

$$g_{2,i} = r_i^2 - (r_i - c_i)^2. \tag{4}$$

Selecting the two largest reductions per quartet forms a_2 from the quantized residual c. It can therefore represent an omitted value or correct the coarse approximation.

The second selection is optimal conditional on a_1 and can recover the complementary mask. At fixed scales in exact arithmetic, the result therefore has no greater activation reconstruction error than the best complementary assignment. This local guarantee does not extend to output error or perplexity.

3.4 Smoothing

Large activation channels can determine the scale of an entire block, reducing the precision available to the remaining values. We use SmoothQuant (Xiao et al., 2023) with a default migration strength $\alpha = 0.5$ to redistribute channel magnitude between activations and weights before quantization. For a positive channel multiplier b, we form

$$U = \mathrm{BF16}(X \odot b), \tag{5}$$
$$W_4 = Q_{\mathrm{FP4,G32,UE5M3}}(W/b)\,. \tag{6}$$

Inverse scaling of the weights preserves the linear transformation before rounding and quantization. We quantize the transformed weights with RTN by default.

3.5 Calibrating the Scale Ratio

Calibration chooses p by layer-output error, accounting for how the fixed weights amplify activation errors. For projections j sharing a raw input, we test $\mathcal{P} = \{0.05, 0.10, \ldots, 1.00\}$, holding b and W_4 fixed. Each candidate reruns scale rounding and both selections on the calibration inputs. We select

$$p^* = \operatorname*{argmin}_{p \in \mathcal{P}} \sum_j \|\Delta U^{(j)}(p)(W_4^{(j)})^\top\|_F^2,$$

$$\Delta U^{(j)}(p) = A_1^{(j)}(p) + A_2^{(j)}(p) - U^{(j)}. \tag{7}$$

Here $U^{(j)}$ is projection j's prepared BF16 input. The reference uses these activations with the same quantized weights, so calibration targets the error introduced by activation encoding.

After calibration, each new input is encoded into two streams, giving the approximate layer output

$$\widehat{Y} = \mathrm{BF16}\left[A_1 W_4^\top + A_2 W_4^\top\right]. \tag{8}$$

Inverse smoothing is absorbed into W_4, so the output needs no rescaling.

4 Execution on Vera Rubin

Adaptive assignment adds online work and metadata traffic. Efficient execution must control these costs while processing both streams without duplicating global-memory weight-tile loads or outputs. We prepare and pack weights offline, with channel transforms and scale ratios fixed before inference. At runtime, a fused preparation kernel encodes the activations, followed by a shared-weight sparse GEMM.

4.1 Fused Activation Preparation

Each new input requires selecting and packing two sparsity patterns. We fuse the channel transform, scale computation, greedy assignment (Section 3.3), and packing into one kernel. The coarse scale uses the transformed input block's maximum; the fine scale is derived using the calibrated ratio, avoiding a second maximum reduction over the residual. The kernel writes FP4 codes, UE5M3 scales, and position metadata while preserving the original K positions, so changing sparsity patterns requires no weight rearrangement.

The FP4 value budget is unchanged, but metadata and scales add traffic: each G32 block stores 16 bytes of values, 8 bytes of position metadata, and 2 bytes of scales. This is 6.5 bits per logical activation before padding and replication, compared with 4.5 bits for dense G16 FP4, excluding any tensor-level factor.

4.2 Shared-Weight Sparse GEMM

Both streams multiply the same weights. Our GEMM shares each weight tile staged in shared memory across both sparse streams, avoiding a separate global-memory weight-tile load for the second stream. Both products update one FP32 accumulator. The epilogue converts the result to BF16 without materializing intermediate products.

Together, the products perform dense GEMM's $2MNK$ nonzero FLOPs. Our latency measurements include both the activation-preparation kernel and the shared-weight GEMM, capturing the cost of assignment and additional metadata traffic (Section 5.2).

For a standard projection, preparation reads the incoming activation tensor. In grouped MoE, preparation for the first expert GEMM follows token dispatch; preparation for the second follows the intermediate activation. Routing and the weighted combination of expert outputs remain unchanged.

5 Evaluation

5.1 Experimental Setup

Models. We begin by evaluating quantization from BF16 on one dense model, Qwen3.5-4B (Qwen Team, 2026a), and one MoE model, Nemotron-3-Nano-30B-A3B (NVIDIA, 2025a). We then study how VR-SparseQuant adapts to natively quantized checkpoints using GPT-OSS-20B (OpenAI, 2025), MiniMax-M3.1-Flash (MiniMax, 2026), and DeepSeek-V4.1-Flash-1017 (DeepSeek-AI, 2026). To test its extension to joint language and vision quantization, we evaluate Qwen3-VL-32B-Instruct (Qwen Team, 2025) and Nemotron-Nano-12B-v2-VL (NVIDIA, 2025b). We use base checkpoints for perplexity and post-trained checkpoints for task accuracy.

Quantization. Unless specified otherwise, four-bit operands use VRFP4: E2M1 values with one UE5M3 scale per block. Dense W4A4 uses G16 weights and activations; VR-SparseQuant uses G32 weights and 32 logical positions per activation stream. W4A16 uses G32 weights and BF16 activations. We test the composability of VR-SparseQuant with vanilla RTN, AWQ, and Four Over Six (Lin et al., 2024; Cook et al., 2025) to evaluate its applicability across quantization methods. Appendix C provides more details on which model components are quantized and which retain reference precision.

5.2 Language-Model Quality and Kernel Efficiency

Two-stream recipes improve FP4 quality. Table 2 tests full-model quality across dense and MoE architectures using WikiText-2 perplexity (Merity et al., 2016), a six-task commonsense reasoning (CSR) average, AIME2026 (Mathematical Association of America, 2025), GPQA Diamond (Rein et al., 2023), and MMLU-Pro (Wang et al., 2024a). Commonsense is the unweighted zero-shot mean of PIQA (Bisk

Table 2. **VR-SparseQuant composes with RTN, AWQ, and Four Over Six.** It lowers perplexity relative to dense W4A4 on both dense and MoE models. PPL uses base checkpoints; accuracy (%) uses post-trained versions. Model-name rows are BF16; W4A16 retains BF16 activations. All FP4 results use VRFP4 with UE5M3 block scales.

Method	Group	WikiText-2 PPL↓	CSR 6-task Avg.↑	AIME2026↑	GPQA-D↑	MMLU-Pro↑
Qwen3.5-4B	—	8.05	70.2	87.8	77.1	79.5
+ RTN-W (W4A16)	32	8.43	69.1	71.1	76.8	77.3
+ RTN	16	8.67	66.6	58.9	72.6	76.6
+ VR-SQ	**32**	**8.53**	**68.2**	**67.8**	**74.6**	**76.9**
+ AWQ (W4A16)	32	8.34	69.5	76.7	70.7	77.1
+ AWQ	16	8.57	67.9	80.0	69.9	77.2
+ VR-SQ	**32**	**8.48**	**68.5**	**77.8**	**71.7**	**77.1**
+ Four Over Six (W4A16)	32	8.39	69.3	74.4	73.9	77.1
+ Four Over Six	16	8.57	68.4	64.4	70.7	75.3
+ VR-SQ	**32**	**8.52**	**68.6**	**65.6**	**69.7**	**77.0**
Nemotron-3-Nano-30B-A3B	—	5.33	69.8	87.8	59.9	72.6
+ RTN-W (W4A16)	32	6.05	69.3	86.7	60.8	68.8
+ RTN	16	6.85	68.5	78.9	52.7	61.6
+ VR-SQ	**32**	**5.57**	**69.4**	**86.7**	**58.8**	**70.5**
+ AWQ (W4A16)	32	5.48	69.4	81.1	62.6	74.9
+ AWQ	16	5.62	68.5	81.1	61.3	71.3
+ VR-SQ	**32**	**5.52**	**68.9**	**82.2**	**60.8**	**72.8**
+ Four Over Six (W4A16)	32	5.97	69.1	78.9	54.7	67.4
+ Four Over Six	16	6.82	69.0	80.0	52.9	64.9
+ VR-SQ	**32**	**5.73**	**69.4**	**83.3**	**62.8**	**69.7**

et al., 2020), ARC-e and ARC-c (Clark et al., 2018), HellaSwag (Zellers et al., 2019), WinoGrande (Sakaguchi et al., 2020), and BoolQ (Clark et al., 2019), a six-task subset of the LLM-Adapters benchmarks (Hu et al., 2023). AIME2026 uses a pinned community transcription of the 30 AIME I/II questions (math-ai, 2026); AIME2026 and GPQA-D average three runs. MMLU-Pro builds on MMLU (Hendrycks et al., 2021); we use one full evaluation pass.

VR-SparseQuant composes with RTN, AWQ, and Four Over Six, lowering perplexity relative to dense W4A4 and recovering much of the corresponding W4A16 performance. On Nemotron-3-Nano-30B-A3B, the RTN and Four Over Six recipes even achieve lower perplexity than their W4A16 references.

RTN recipe recovers reasoning accuracy. On Qwen3.5-4B, VR-SparseQuant raises AIME2026 accuracy from 58.9% to 67.8% and GPQA-D from 72.6% to 74.6% relative to dense RTN W4A4. On Nemotron, VR-SparseQuant raises AIME2026 accuracy from 78.9% to 86.7%, matching the W4A16 reference, and GPQA-D from 52.7% to 58.8%.

Native kernels accelerate GEMM and MoE over BF16. Figure 5 compares BF16, W4A16, dense W4A4, and our sparse W4A4 on an MLP gate/up GEMM (TP1) and a routed MoE FFN (TP2). For dense W4A4, we select the fastest applicable state-of-the-art kernel evaluated per shape, including FlashInfer's `mm_fp4` (Ye et al., 2025; FlashInfer Contributors, b) and the `flashinfer_cutedsl` and `trtllm_moe` backends (FlashInfer Contributors, a;c). Across input sizes from 1 to 2,048, VR-SparseQuant achieves 1.7–3.1× speedup over BF16 for GEMM and 1.4–2.7× for MoE. Relative to these optimized W4A4 baselines, VR-SparseQuant has 7–27% higher latency for the tested GEMMs with $M \geq 16$ and 18–47% higher latency for the tested MoE sizes. Timings include online activation preparation but exclude weight preparation and communication; the MoE benchmark uses precomputed balanced routing.

5.3 Quantizing Full and Linear Attention

Attention projections can be sensitive to quantization. We test whether VR-SparseQuant improves FP4 quality in both full-attention and linear-attention layers, extending its benefit beyond MLPs. Table 3 starts with 96 MLP projections in Qwen3.5-4B-Base, then adds 32 full-attention and 120 Gated DeltaNet projections. Each recipe retains its fitted weights as coverage expands; excluded projections remain BF16. With full attention included, VR-SparseQuant lowers perplexity from 8.50 to 8.39 relative to dense RTN W4A4; including linear attention lowers it from 8.67 to 8.53. The smaller perplexity increase at each step shows that VR-SparseQuant reduces the quality penalty of quantizing both attention families.

5.4 Applying VR-SparseQuant to Pre-Quantized Models

We also evaluate VR-SparseQuant on models released with quantized weights, including GPT-OSS-20B, MiniMax-M3.1-Flash, and DeepSeek-V4.1-Flash-1017. Their native

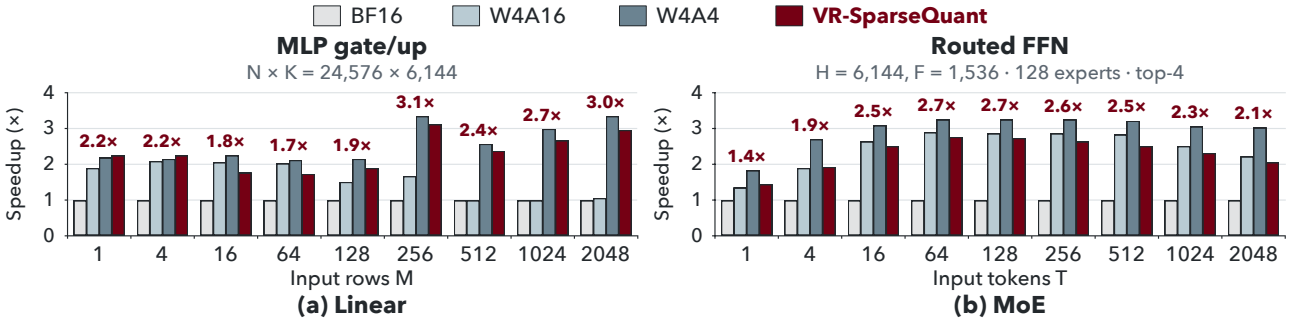


Figure 5. **Operator speedup and the cost of sparse encoding on Rubin.** BF16 median latency divided by method median latency for (a) an MLP gate/up GEMM at TP1 and (b) a routed FFN at TP2, using synthetic inputs with MiniMax shapes. Timings include online activation preparation and exclude weight preparation and communication; MoE routes are precomputed and balanced. W4A4 uses the fastest applicable kernel for each shape. GEMM uses a PyTorch BF16 baseline; MoE uses TRT-LLM BF16.

Table 3. **VR-SparseQuant reduces the perplexity penalty of quantizing attention.** Cumulative coverage on Qwen3.5-4B-Base extends from MLPs to full attention and Gated DeltaNet linear attention.

Coverage	# Proj.	RTN W4A4 G16	VR-SQ W4A4 G32	RTN-W W4A16 G32
MLP	96	8.39	**8.33**	8.25
+ Full attn.	128	8.50	**8.39**	8.30
+ Linear attn.	248	8.67	**8.53**	8.43

Table 4. **VR-SparseQuant improves GPT-OSS-20B GPQA-D accuracy over dense FP4 under both scale formats.** Decoded expert weights remain unchanged.

Method	Scaling Type / Group Size			GPQA-D↑
	W	A	G	
W4A16	UE8M0	—	32	71.7
W4A4	UE8M0	UE8M0	32	69.4
+ VR-SQ	UE8M0	UE8M0	32	**70.9**
W4A4	UE5M3	UE5M3	16	69.5
+ VR-SQ	UE5M3	UE5M3	32	**70.7**

differences in group size and scale format provide a broad range of settings for testing VR-SparseQuant.

GPT-OSS-20B. GPT-OSS-20B stores its expert weights in MXFP4 with G32, matching the weight-group size required by our sparse Tensor Core path. We can therefore apply VR-SparseQuant without requantizing the weights. The first W4A4 pair in Table 4 retains the native UE8M0 scales and shows improved GPQA-D accuracy with VR-SparseQuant. All weight scales used by this checkpoint are also exactly representable in UE5M3, allowing us to recode them into VRFP4 without changing the E2M1 codes or decoded weights. UE5M3 also permits finer activation-scale choices than power-of-two UE8M0 scaling. In both formats, VR-SparseQuant recovers much of the GPQA-D accuracy lost when activations are quantized to FP4. With UE8M0 scales, it reaches 70.9% accuracy, compared with 71.7% for W4A16.

For MiniMax and DeepSeek, we apply VR-SparseQuant first to the experts alone and then to all eligible linear projections (Table 5). We evaluate BFCL v4 (Patil et al., 2025; Mao et al., 2025), τ^2-Bench (Barres et al., 2025), AutomationBench (Shepard & Salimans, 2026), and CharXiv reasoning questions (RQ) (Wang et al., 2024b). BFCL scores average four blocks; τ^2-Bench scores average domain pass[1]. We use GPT-5.5 to judge CharXiv and maximum reasoning effort for all runs, keeping evaluation settings fixed within each model.

MiniMax-M3.1-Flash. MiniMax uses NVFP4 G16 for its experts and MXFP8 for other linear projections (MiniMax, 2026; NVIDIA, 2026a). For VR-SparseQuant, we requantize the decoded expert weights to VRFP4 G32 with RTN. This conversion merges G16 groups and folds NVFP4's global factor into UE5M3 block scales, adding some weight-quantization error. The expert-only variant outperforms the native checkpoint on BFCL and CharXiv, achieving the highest CharXiv score among the tested MiniMax variants (Table 5). When applied to all eligible linear projections, VR-SparseQuant raises AutomationBench from 61.5% to 62.7%.

DeepSeek-V4.1-Flash-1017. DeepSeek's native checkpoint uses MXFP4 G32 expert weights and MXFP8 G32 activations (DeepSeek-AI, 2026; NVIDIA, 2026e). On the expert projections, VR-SparseQuant improves three of four tasks over dense MXFP4 activation quantization. Applying VR-SparseQuant to all eligible linear projections brings both weights and activations to VRFP4 and reaches **74.3% BFCL** and **90.4% τ^2-Bench**, compared with 73.0% and 88.2% for the native checkpoint. Quantizing the MoE activations to FP4 also moves these projections from the native W4A8 format to W4A4, allowing our kernels to use Rubin's higher-throughput compact FP4 Tensor Core path (NVIDIA, 2026g;h). This setting includes shared-expert, attention, indexer, and dense feed-forward projections; other operators, caches, and states retain reference precision.

Table 5. **Applying VR-SparseQuant to natively quantized models.** Relative to the native checkpoints, expert-only MiniMax improves on three of four tasks, and all-linear DeepSeek improves BFCL and τ^2-Bench. W/A denote weight/activation formats; G is group size. VRFP4 uses UE5M3 block scales; NVFP4 combines FP8 block and FP32 global scales. Scores are percentages. † DeepSeek's native FP8 linear weights use 32×32 tiles; activations use G32.

Method	Experts			Linears			BFCL v4 Avg.↑	τ^2-Bench Avg.↑	Automation Bench↑	CharXiv RQ↑
	W	A	G	W	A	G				
MiniMax-M3.1-Flash										
Native	NVFP4	NVFP4	16	MXFP8	MXFP8	32	64.3	84.0	61.5	82.3
+ VR-SQ	VRFP4	VRFP4	32	MXFP8	MXFP8	32	**64.9**	**84.6**	**59.2**	**82.6**
VRFP4 (all)	VRFP4	VRFP4	16	VRFP4	VRFP4	16	65.0	83.9	62.5	81.5
+ VR-SQ	VRFP4	VRFP4	32	VRFP4	VRFP4	32	**64.5**	**83.7**	**62.7**	**81.5**
DeepSeek-V4.1-Flash-1017										
Native	MXFP4	MXFP8	32	FP8	MXFP8	32†	73.0	88.2	60.3	77.9
MXFP4 (experts)	MXFP4	MXFP4	32	FP8	MXFP8	32†	73.6	87.9	58.7	77.0
+ VR-SQ	VRFP4	VRFP4	32	FP8	MXFP8	32†	**73.9**	**87.7**	**59.0**	**77.7**
VRFP4 (linears)	MXFP4	MXFP4	32	VRFP4	VRFP4	16	73.9	89.1	58.8	75.6
+ VR-SQ	VRFP4	VRFP4	32	VRFP4	VRFP4	32	**74.3**	**90.4**	**59.2**	**75.5**

5.5 Vision-Language Quality

Joint backbone quantization tests the representation across language and visual activation distributions. We evaluate MMMU (Yue et al., 2024), DocVQA (Mathew et al., 2021), TextVQA (Singh et al., 2019), and ChartQA (Masry et al., 2022) and report their unweighted mean, using ANLS for DocVQA. The multimodal connector and visual-token settings remain fixed. Our TinyChat baseline combines AWQ W4A16 G16 language weights with dynamic E4M3 W8A8 vision quantization (Lin et al., 2024; Yang et al., 2024).

We combine channel smoothing with AWQ G32 weights in both backbones. VR-SparseQuant improves the average score over dense RTN W4A4 on both models (Table 6), while executing the quantized language and vision projections directly on Rubin's FP4 Tensor Cores.

5.6 Ablation Studies

These studies separate activation encoding from smoothing and examine scale ratios, sparsity patterns, and weight-group size. Unless stated otherwise, quality studies use Qwen3.5-4B-Base WikiText-2 perplexity with fixed calibration data, projections, and evaluation tokens. Hyperparameters are selected on calibration data alone.

Encoding improves quality before smoothing. With fixed RTN G32 weights, no smoothing, and 248 projections, Table 7 isolates activation encoding: replacing dense FP4 activations with two streams lowers perplexity from 8.86 to 8.58. Rebuilding RTN weights and recalibrating p after smoothing, under the same budget, reduces it further to 8.53.

Different projections favor different scale ratios. The fine scale balances resolution of small residuals against coverage of larger values. For the weighted-assignment variant in Figure 6, input projections favor $p = 0.85$, while output projections favor 0.40–0.60. The different minima motivate selecting a ratio for each shared-input group through the output-error objective in Section 3.5.

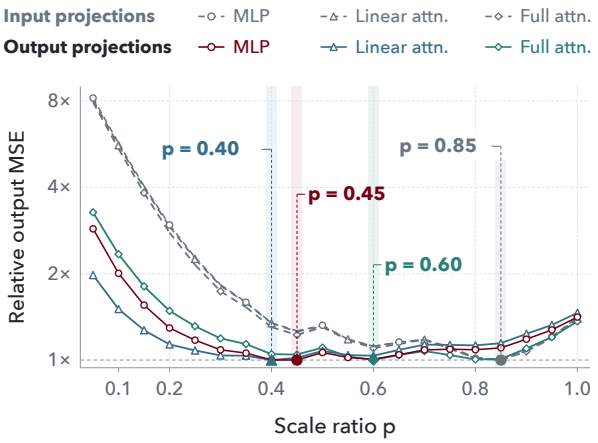


Figure 6. **Different projections favor different scale ratios.** Scale-ratio study of a weighted-assignment variant on Qwen3.5-4B. Output MSE is pooled by projection type and normalized by its minimum over 20 ratios (log axis). Dashed/solid curves denote input/output projections; filled markers identify minima. These curves motivate separate calibration for different projections.

Residual correction competes with complete coverage. Table 8 compares complete coverage with residual correction using identical smoothed RTN G32 weights. Complementary splitting assigns the two largest magnitudes per quartet to one stream and the remaining values to the other, with independent scales. It gives 8.73 PPL, versus 8.53 for VR-SparseQuant's calibrated ratio and error-based assignment (Eqs. 3–4). This gain measures assignment and scaling together, not overlap alone.

Figure 7 shows the mechanism at fixed scales: moving one

Table 6. **FP4 quantization of both language and vision backbones.** VR-SparseQuant improves the mean over dense RTN W4A4 and stays within 0.7 points of TinyChat while using native FP4 Tensor Core execution. DocVQA uses ANLS. † Our TinyChat implementation combines dynamic FP8 vision quantization with AWQ-style W4A16 language quantization.

Method	Precision		MMMU	DocVQA	TextVQA	ChartQA	Avg.↑
	Language model	Vision encoder					
Qwen3-VL-32B-Instruct	BF16	BF16	62.7	96.1	83.5	87.1	82.4
+ RTN	W4A4	W4A4	61.8	95.6	81.1	86.3	81.2
+ TinyChat†	W4A16	W8A8	63.2	95.9	83.0	86.7	82.2
+ VR-SQ	W4A4	W4A4	**62.8**	**96.1**	**82.1**	**86.1**	**81.8**
Nemotron-Nano-12B-v2-VL	BF16	BF16	64.6	94.3	82.8	89.6	82.8
+ RTN	W4A4	W4A4	63.1	93.4	82.0	89.0	81.9
+ TinyChat†	W4A16	W8A8	65.6	94.1	82.8	89.2	82.9
+ VR-SQ	W4A4	W4A4	**63.3**	**94.1**	**82.0**	**89.6**	**82.2**

Table 7. **Two-stream encoding lowers perplexity before smoothing.** The smoothed recipe reduces it further.

Method	Group	Smoothing	Sparsity	WikiText-2 PPL↓
RTN-W (W4A16)	32	—	—	8.43
RTN W4A4	32	—	—	8.86
RTN W4A4	16	—	—	8.67
VR-SQ W4A4	32	×	✓	8.58
VR-SQ W4A4	32	✓	✓	8.53

Group size applies to quantized operands. Smoothing uses $\alpha = 0.5$, rebuilt RTN weights, and recalibrated p. Dashes in the component columns mean not applied.

Table 8. **Assignment and scale calibration.** VR-SparseQuant gives lower perplexity than complementary splitting with the same smoothed RTN G32 weights. Complementary uses independent stream scales; VR-SparseQuant uses calibrated p. Schematic masks show coarse above fine.

Assignment	Masks	WikiText-2 PPL↓
Complementary		8.73
VR-SQ		**8.53**

fine-stream slot from a small value to a residual correction reduces the constructed quartet's MSE by 9×, with the coarse stream unchanged. Correcting one position can therefore remove more error than omitting another introduces.

Table 9. **Elementwise 2:4 sparsity lowers perplexity relative to paired 4:8 and approaches flexible 16:32.** Simulated Qwen3.5-4B-Base comparisons with shared G32 VRFP4 weights, smoothing, and calibrated p across sparse variants.

Structure	Selection per stream	WikiText-2 PPL↓
Dense Quantization (Control)	All 32 values	8.86
Paired 4:8 (Blackwell)	2 of 4 adjacent pairs	8.61
Elementwise 2:4 (Vera Rubin)	Any 2 of 4 values	**8.53**
Flexible 16:32 (Oracle)	Any 16 of 32 values	8.49

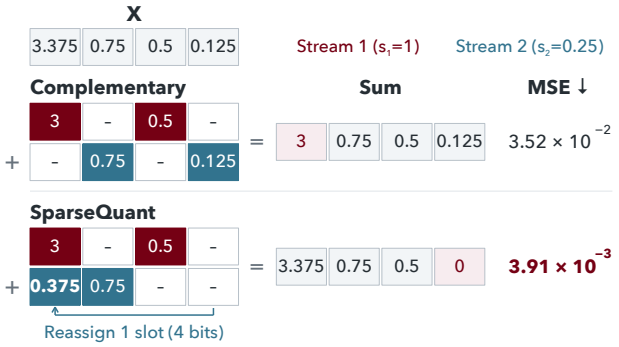


Figure 7. **Trading coverage for correction.** In this constructed quartet, reallocating one fine-stream slot to a residual reduces reconstruction MSE by 9×. Both encodings retain two values per stream at the same scales, with VR-SparseQuant's coarse selection fixed. This comparison isolates overlap; Table 8 also changes the scale rule.

Elementwise structured sparsity improves representation quality. Paired 4:8 couples neighboring values; Vera Rubin's elementwise 2:4 selects individual values per quartet (Figure 2). Table 9 tests simulated quality with fitted G32 weights, smoothing, and p fixed across sparse variants. Elementwise selection lowers perplexity from 8.61 to 8.53; allowing any 16 positions per block with the same sequential greedy rule, rather than exhaustive search, reaches 8.49. Thus, 2:4 captures most of the improvement over paired selection, with a further gain from relaxation. Neither hardware pattern family contains the other; both are subsets of 16:32.

Table 10. **Weight quantization at G16 and G32.** Increasing the weight block size from 16 to 32 incurs a smaller perplexity penalty with AWQ than with RTN.

Quantizer	G16 PPL↓	G32 PPL↓	Relative increase↓
RTN	8.37	8.43	+0.76%
AWQ	8.31	8.34	+0.33%

AWQ reduces the weight-group penalty. To measure the cost of the sparse path's G32 weights, Table 10 retains BF16 activations and varies weight-group size within each quantizer. Increasing G16 to G32 raises perplexity by 0.76% for RTN and 0.33% for AWQ. Both quantize the same 248 projections with E2M1/UE5M3 and no additional smoothing; AWQ retains its channel scaling (Lin et al., 2024). AWQ reduces this weight-side cost, complementing the improvements in activation encoding.

6 Related Work

Dense low-bit quantization. SmoothQuant transfers activation dynamic range to weights, while AWQ protects sensitive weight channels using activation statistics (Xiao et al., 2023; Lin et al., 2024). QuaRot and SpinQuant use rotations, and FlatQuant learns affine transformations, to improve dense weight and activation quantization (Ashkboos et al., 2024; Liu et al., 2025; Sun et al., 2025). For microscaled FP4, Four Over Six selects between two block scales, and MR-GPTQ combines format-specific quantization with blockwise rotations (Cook et al., 2025; Egiazarian et al., 2026). These methods improve dense quantizers' coordinates or grids. VR-SparseQuant instead allocates FP4 contributions across positions through sparse metadata and remains complementary to weight quantization and channel smoothing.

Structured sparse decomposition. TASD approximates weights and activations with multiple structured sparse tensors and sums their products (Jeong et al., 2025). SDQ decomposes weights into structured sparse components with different precisions and error-aware selection (Jeong et al., 2024). VR-SparseQuant's two FP4 activation components can overlap, letting one value correct another's rounding error. Their combined value count equals dense FP4's, so correction competes with representing distinct positions within a fixed allocation.

Residual correction for quantization. SVDQuant absorbs outliers into a high-precision low-rank branch and quantizes the residual to four bits (Li et al., 2025). ARCQuant adds FP4 activation-residual channels to the reduction dimension of a single GEMM (Meng et al., 2026a). SharQ uses a sparse FP4 backbone and a dense FP4 residual, sharing one weight payload across the two products (Meng et al., 2026b). Rollout-ResQ adds a 2:4 FP4 residual to dense FP4 activations for reinforcement-learning rollouts (Mak et al., 2026). VR-SparseQuant constrains both components to 2:4 sparsity. Its residual stream must choose between covering omitted values and correcting represented ones; a calibrated scale ratio and online error-based selection make this choice without searching scale pairs at inference.

Sparse execution. Haziza et al. exploit Squared-ReLU activation sparsity with online 2:4 packing and fused FP8 quantization (Haziza et al., 2025). Attention and token sparsity operate at a different level, reducing the tokens or spatial locations processed (Xiao et al., 2024; Yang et al., 2025; Liu et al., 2024; Khaki et al., 2025). For low-bit execution, TinyChat, QServe, and Nunchaku control dequantization, data movement, and branch overhead (Lin et al., 2024; 2025; Li et al., 2025). ARCQuant and SharQ integrate residual preparation with FP4 execution (Meng et al., 2026a;b). VR-SparseQuant reuses weight tiles and an accumulator for both sparse products in one GEMM on Rubin's compact elementwise FP4 path (NVIDIA, 2026g). SpAtten (Wang et al., 2021) proposes KV Cache sparsification and quantization, we well as the combinations of them.

7 Limitations

VR-SparseQuant improves activation precision through additional encoding work and metadata traffic, which can increase latency relative to dense W4A4. The sparse FP4 Tensor Core path we use requires G32 weights, sacrificing scaling granularity relative to the dense G16 baseline. This can introduce additional weight quantization error that activation encoding cannot directly correct. Stronger weight quantizers are therefore particularly important for mitigating this penalty and retaining the gains from activation encoding (Table 10). Reported speedups cover GEMM and MoE operators, excluding communication and other serving overheads. Native kernel measurements use Vera Rubin; the sparsity-pattern comparisons assess simulated quality. Results for complete recipes and converted checkpoints reflect both weight and activation effects, while the fixed-weight ablations isolate activation encoding.

8 Conclusion

To the best of our knowledge, this is the first study to design and evaluate adaptive activation quantization using Vera Rubin's compact elementwise 2:4 sparse FP4 instructions. VR-SparseQuant uses sparsity patterns to allocate precision: two overlapping streams refine selected activations while retaining dense FP4's value count and nonzero arithmetic. Offline calibration, greedy assignment, and a shared-weight GEMM make this allocation practical. Our kernels use Vera Rubin's direct UE5M3 block scaling for E2M1 FP4, which we term VRFP4.

Across RTN, AWQ, and Four Over Six, VR-SparseQuant improves perplexity relative to dense W4A4, with further applications to vision-language models and released low-bit checkpoints. Including online activation preparation, native Rubin kernels achieve up to $3.1\times$ GEMM and $2.7\times$ MoE speedup over BF16.

More broadly, Rubin's elementwise selection and VRFP4 scaling show how new hardware capabilities expand the design space of quantization. More flexible sparsity patterns, lower metadata costs, and tighter fusion of encoding

with surrounding operators could make adaptive precision allocation more efficient. These results motivate developing quantization formats, algorithms, and kernels together as GPU hardware evolves, with model quality, memory traffic, and latency as joint design objectives.

REFERENCES

Ashkboos, S., Mohtashami, A., Croci, M. L., Li, B., Cameron, P., Jaggi, M., Alistarh, D., Hoefler, T., and Hensman, J. QuaRot: Outlier-free 4-bit inference in rotated LLMs. In *Advances in Neural Information Processing Systems*, volume 37, 2024. doi: 10.52202/079017-3180. URL https://papers.nips.cc/paper_files/paper/2024/hash/b5b939436789f76f08b9d0da5e81af7c-Abstract-Conference.html.

Barres, V., Dong, H., Ray, S., Si, X., and Narasimhan, K. τ^2-Bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*, 2025. URL https://arxiv.org/abs/2506.07982.

Bisk, Y., Zellers, R., Bras, R. L., Gao, J., and Choi, Y. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pp. 7432–7439, 2020. doi: 10.1609/aaai.v34i05.6239. URL https://ojs.aaai.org/index.php/AAAI/article/view/6239.

Clark, C., Lee, K., Chang, M.-W., Kwiatkowski, T., Collins, M., and Toutanova, K. BoolQ: Exploring the surprising difficulty of natural yes/no questions. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 2924–2936, 2019. doi: 10.18653/v1/N19-1300. URL https://aclanthology.org/N19-1300/.

Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., and Tafjord, O. Think you have solved question answering? try ARC, the AI2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018. URL https://arxiv.org/abs/1803.05457.

Cook, J., Guo, J., Xiao, G., Lin, Y., Wyss, K., Nazemi, M., Mishra, A., del Mundo, C., Blankevoort, T., and Han, S. Four over six: More accurate NVFP4 quantization with adaptive block scaling. *arXiv preprint arXiv:2512.02010*, 2025. URL https://arxiv.org/abs/2512.02010.

Darvish Rouhani, B., Zhao, R., More, A., Hall, M., Khodamoradi, A., Deng, S., Choudhary, D., Cornea, M., Dellinger, E., Denolf, K., Stosic, D., Elango, V., Golub, M., Heinecke, A., James-Roxby, P., Jani, D., Kolhe, G., Langhammer, M., Li, A., Melnick, L., Mesmakhosroshahi, M., Rodriguez, A., Schulte, M., Shafipour, R., Shao, L., Siu, M., Dubey, P., Micikevicius, P., Naumov, M., Verrilli, C., Wittig, R., Burger, D., and Chung, E. Microscaling data formats for deep learning. *arXiv preprint arXiv:2310.10537*, 2023. URL https://arxiv.org/abs/2310.10537.

DeepSeek-AI. DeepSeek-V4.1-Flash-1017: Reference inference implementation. Hugging Face model repository, 2026. URL https://huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash-1017/blob/main/inference/model.py. Accessed October 24, 2026.

Egiazarian, V., Castro, R. L., Kuznedelev, D., Panferov, A., Kurtić, E., Pandit, S., Marques, A., Kurtz, M., Ashkboos, S., Hoefler, T., and Alistarh, D. Bridging the gap between promise and performance for microscaling FP4 quantization. In *International Conference on Learning Representations*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/b87bb4f6346d727b265088235e5bc389-Abstract-Conference.html.

FlashInfer Contributors. *flashinfer.fused_moe.cute_dsl_fused_moe_nvfp4*. FlashInfer, a. URL https://docs.flashinfer.ai/generated/flashinfer.fused_moe.cute_dsl_fused_moe_nvfp4.html. FlashInfer 0.7.2.post1 API documentation. Accessed October 30, 2026.

FlashInfer Contributors. *flashinfer.gemm.mm_fp4*. FlashInfer, b. URL https://docs.flashinfer.ai/generated/flashinfer.gemm.mm_fp4.html. FlashInfer 0.7.2.post1 API documentation. Accessed October 30, 2026.

FlashInfer Contributors. *flashinfer.fused_moe.trtllm_fp4_block_scale_moe*. FlashInfer, c. URL https://docs.flashinfer.ai/generated/flashinfer.fused_moe.trtllm_fp4_block_scale_moe.html. FlashInfer 0.7.2.post1 API documentation. Accessed October 30, 2026.

Haziza, D., Chou, T., Choudhary, D., Cai, J., Wehrstedt, L., Massa, F., Yu, J., Jeong, G., Rao, S., and Labatut, P. Accelerating transformer inference and training with 2:4 activation sparsity. In *ICLR 2025 Workshop on Sparsity in LLMs (SLLM)*, 2025. URL https://openreview.net/pdf?id=O5feVk7p6Y.

Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. Measuring massive multitask language understanding. In *International Conference*

on Learning Representations, 2021. URL https://arxiv.org/abs/2009.03300.

Hu, Z., Wang, L., Lan, Y., Xu, W., Lim, E.-P., Bing, L., Xu, X., Poria, S., and Lee, R. LLM-Adapters: An adapter family for parameter-efficient fine-tuning of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 5254–5276. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.emnlp-main.319. URL https://aclanthology.org/2023.emnlp-main.319/.

Jeong, G., Tsai, P.-A., Keckler, S. W., and Krishna, T. SDQ: Sparse decomposed quantization for LLM inference. *arXiv preprint arXiv:2406.13868*, 2024. URL https://arxiv.org/abs/2406.13868.

Jeong, G., Tsai, P.-A., Bambhaniya, A. R., Keckler, S. W., and Krishna, T. Enabling unstructured sparse acceleration on structured sparse accelerators. In *Proceedings of Machine Learning and Systems*, volume 7, 2025. URL https://proceedings.mlsys.org/paper_files/paper/2025/hash/e2ec2530db26b54d0b3b060c1e4a1bda-Abstract-Conference.html.

Khaki, S., Guo, J., Tang, J., Yang, S., Chen, Y., Plataniotis, K. N., Lu, Y., Han, S., and Liu, Z. SparseVILA: Decoupling visual sparsity for efficient VLM inference. *arXiv preprint arXiv:2510.17777*, 2025. URL https://arxiv.org/abs/2510.17777.

Li, M., Lin, Y., Zhang, Z., Cai, T., Li, X., Guo, J., Xie, E., Meng, C., Zhu, J.-Y., and Han, S. SVDQuant: Absorbing outliers by low-rank components for 4-bit diffusion models. In *International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2411.05007.

Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, X., Gan, C., and Han, S. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In *Proceedings of Machine Learning and Systems*, volume 6, 2024. URL https://arxiv.org/abs/2306.00978.

Lin, Y., Tang, H., Yang, S., Zhang, Z., Xiao, G., Gan, C., and Han, S. QServe: W4A8KV4 quantization and system co-design for efficient LLM serving. In *Proceedings of Machine Learning and Systems*, volume 7, 2025. URL https://arxiv.org/abs/2405.04532.

Liu, Z., Zhang, Z., Khaki, S., Yang, S., Tang, H., Xu, C., Keutzer, K., and Han, S. Sparse refinement for efficient high-resolution semantic segmentation. In *European Conference on Computer Vision*, 2024. URL https://arxiv.org/abs/2407.19014.

Liu, Z., Zhao, C., Fedorov, I., Soran, B., Choudhary, D., Krishnamoorthi, R., Chandra, V., Tian, Y., and Blankevoort, T. SpinQuant: LLM quantization with learned rotations. In *International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/e5b1c0d4866f72393c522c8a00eed4eb-Abstract-Conference.html.

Mak, H. Y., Golestan, S., Le, H., Jazi, M. T., Peng, Y., Wang, Y., Wang, Y., Wang, J., Hu, T., He, F., Hu, G., Rahman, T., and Raju, A. D. HiFloat4 format for end-to-end reinforcement learning post-training of large language models. *arXiv preprint arXiv:2607.26515*, 2026. URL https://arxiv.org/abs/2607.26515.

Mao, H., Tsao, R., Zhou, J., Patil, S. G., and Gonzalez, J. E. BFCL V4: Web search. Official Berkeley Function Calling Leaderboard documentation, July 2025. URL https://gorilla.cs.berkeley.edu/blogs/15_bfcl_v4_web_search.html. Released July 17, 2025; updated July 20, 2025. Includes V4 score composition.

Masry, A., Long, D. X., Tan, J. Q., Joty, S., and Hoque, E. ChartQA: A benchmark for question answering about charts with visual and logical reasoning. In *Findings of the Association for Computational Linguistics: ACL 2022*, pp. 2263–2279. Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.findings-acl.177. URL https://aclanthology.org/2022.findings-acl.177/.

math-ai. aime26. Hugging Face dataset, 2026. URL https://huggingface.co/datasets/math-ai/aime26/tree/79037aebdb6580008fb960d17cb21fd3099083e3. Community transcription of AIME 2026 I and II. Revision 79037aebdb6580008fb960d17cb21fd3099083e3. Accessed October 30, 2026.

Mathematical Association of America. 2025–26 AIME thresholds are now available. Official competition announcement, December 2025. URL https://maa.org/news/2025-26-aime-thresholds-are-now-available/. Published December 15, 2025; announces AIME I and II in February 2026.

Mathew, M., Karatzas, D., and Jawahar, C. V. DocVQA: A dataset for vqa on document images. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2021. URL https://arxiv.org/abs/2007.00398.

Meng, H., Luo, Y., Zhao, Y., Liu, W., Zhang, P., and Ma, X. ARCQuant: Boosting NVFP4 quantization

with augmented residual channels for LLMs. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 8609–8623, San Diego, California, United States, 2026a. Association for Computational Linguistics. doi: 10.18653/v1/2026.acl-long.388. URL https://aclanthology.org/2026.acl-long.388/.

Meng, H., Luo, Y., Zhao, Y., Liu, W., Zheng, H., Ma, X., and Zhang, P. SharQ: Bridging activation sparsity and FP4 quantization for LLM inference. *arXiv preprint arXiv:2606.26587*, 2026b. URL https://arxiv.org/abs/2606.26587.

Merity, S., Xiong, C., Bradbury, J., and Socher, R. Pointer sentinel mixture models, 2016. URL https://arxiv.org/abs/1609.07843.

MiniMax. MiniMax-M3.1-Flash: Released quantization configuration. Hugging Face model repository, 2026. URL https://huggingface.co/MiniMaxAI/MiniMax-M3.1-Flash/blob/main/config.json. Accessed October 24, 2026.

NVIDIA. *FP8 Blockwise Scaling*. NVIDIA Transformer Engine. URL https://nvidia.github.io/TransformerEngine/features/low_precision_training/fp8_blockwise_scaling/fp8_blockwise_scaling.html. Transformer Engine 2.22.0.dev0 documentation. Accessed October 30, 2026.

NVIDIA. NVIDIA-Nemotron-3-Nano-30B-A3B-BF16: Model card. Hugging Face model repository, 2025a. URL https://huggingface.co/nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B-BF16. Released December 15, 2025. Accessed September 30, 2026.

NVIDIA. NVIDIA Nemotron Nano 12B V2 VL BF16: Model card. Hugging Face model repository, 2025b. URL https://huggingface.co/nvidia/NVIDIA-Nemotron-Nano-12B-v2-VL-BF16. Accessed October 26, 2026.

NVIDIA. *Parallel Thread Execution ISA, Version 8.7*, 2025c. URL https://docs.nvidia.com/cuda/archive/12.8.1/parallel-thread-execution/index.html. Sparse block scaling and FP4 metadata; accessed October 20, 2026.

NVIDIA. MiniMax-M3.1-Flash: Native inference formats. NVIDIA NIM model card, 2026a. URL https://build.nvidia.com/minimaxai/minimax-m3.1-flash/modelcard. Accessed October 24, 2026.

NVIDIA. *NVIDIA A100 Tensor Core GPU Datasheet*, 2026b. URL https://images.nvidia.com/data-center/a100/a100-datasheet.pdf. 80GB SXM specifications; accessed October 20, 2026.

NVIDIA. *NVIDIA Blackwell Architecture Datasheet*, 2026c. URL https://dam-cdn.nvd.orangelogic.com/AssetLink/y441155802qub41q118b2852i557jem5.pdf. Individual HGX B200 GPU specifications, p. 8; accessed October 20, 2026.

NVIDIA. *NVIDIA H100 Tensor Core GPU Specifications*, 2026d. URL https://www.nvidia.com/en-us/data-center/h100/. SXM specifications; accessed October 20, 2026.

NVIDIA. *MXFP8*. NVIDIA, 2026e. URL https://docs.nvidia.com/deeplearning/transformer-engine/features/low_precision_training/mxfp8/mxfp8.html. Transformer Engine documentation. Accessed October 24, 2026.

NVIDIA. *NVFP4: Two-Level Scaling*. NVIDIA Transformer Engine, 2026f. URL https://nvidia.github.io/TransformerEngine/features/low_precision_training/nvfp4/nvfp4.html. Accessed October 5, 2026.

NVIDIA. *Parallel Thread Execution ISA, Version 9.4*. NVIDIA, 2026g. URL https://docs.nvidia.com/cuda/parallel-thread-execution/index.html. Accessed October 5, 2026.

NVIDIA. *NVIDIA HGX: Individual Rubin GPU Specifications*, 2026h. URL https://www.nvidia.com/en-us/data-center/hgx/. Sparse NVFP4 inference specification; accessed October 20, 2026.

NVIDIA. *NVIDIA Vera Rubin Platform*, 2026i. URL https://www.nvidia.com/en-us/data-center/technologies/rubin/. Accessed October 29, 2026.

NVlabs. Kernel Design Agents (KDA). GitHub repository, 2026. URL https://github.com/NVlabs/kda. Accessed October 30, 2026.

OpenAI. gpt-oss-120b and gpt-oss-20b model card, 2025. URL https://arxiv.org/abs/2508.10925.

Patil, S. G., Mao, H., Yan, F., Ji, C. C.-J., Suresh, V., Stoica, I., and Gonzalez, J. E. The berkeley function calling leaderboard (BFCL): From tool use to

agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 48371–48392. PMLR, 2025. URL https://proceedings.mlr.press/v267/patil25a.html.

Qwen Team. Qwen3-VL-32B-Instruct: Model card. Hugging Face model repository, 2025. URL https://huggingface.co/Qwen/Qwen3-VL-32B-Instruct. Accessed September 30, 2026.

Qwen Team. Qwen3.5-4B: Model card. Hugging Face model repository, 2026a. URL https://huggingface.co/Qwen/Qwen3.5-4B. Accessed September 30, 2026.

Qwen Team. Qwen3.5-4B-Base: Model card. Hugging Face model repository, 2026b. URL https://huggingface.co/Qwen/Qwen3.5-4B-Base. Accessed October 30, 2026.

Rein, D., Hou, B. L., Stickland, A. C., Petty, J., Pang, R. Y., Dirani, J., Michael, J., and Bowman, S. R. GPQA: A graduate-level google-proof Q&A benchmark. *arXiv preprint arXiv:2311.12022*, 2023. URL https://arxiv.org/abs/2311.12022.

Sakaguchi, K., Bras, R. L., Bhagavatula, C., and Choi, Y. WinoGrande: An adversarial winograd schema challenge at scale. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pp. 8732–8740, 2020. doi: 10.1609/aaai.v34i05.6399. URL https://ojs.aaai.org/index.php/AAAI/article/view/6399.

Shepard, D. and Salimans, R. AutomationBench. *arXiv preprint arXiv:2604.18934*, 2026. URL https://arxiv.org/abs/2604.18934.

Singh, A., Natarajan, V., Shah, M., Jiang, Y., Chen, X., Batra, D., Parikh, D., and Rohrbach, M. Towards VQA models that can read. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 8317–8326, 2019. URL https://openaccess.thecvf.com/content_CVPR_2019/html/Singh_Towards_VQA_Models_That_Can_Read_CVPR_2019_paper.html.

Sun, Y., Liu, R., Bai, H., Bao, H., Zhao, K., Li, Y., Hu, J., Yu, X., Hou, L., Yuan, C., Jiang, X., Liu, W., and Yao, J. FlatQuant: Flatness matters for LLM quantization. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 57587–57613. PMLR, 2025. URL https://proceedings.mlr.press/v267/sun25l.html.

Wang, H., Zhang, Z., and Han, S. Spatten: Efficient sparse attention architecture with cascade token and head pruning. In *2021 IEEE international symposium on high-performance computer architecture (HPCA)*, pp. 97–110. IEEE, 2021.

Wang, Y., Ma, X., Zhang, G., Ni, Y., Chandra, A., Guo, S., Ren, W., Arulraj, A., He, X., Jiang, Z., Li, T., Ku, M., Wang, K., Zhuang, A., Fan, R., Yue, X., and Chen, W. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. In *Advances in Neural Information Processing Systems*, 2024a. URL https://arxiv.org/abs/2406.01574.

Wang, Z., Xia, M., He, L., Chen, H., Liu, Y., Zhu, R., Liang, K., Wu, X., Liu, H., Malladi, S., Chevalier, A., Arora, S., and Chen, D. CharXiv: Charting gaps in realistic chart understanding in multimodal LLMs. In *Advances in Neural Information Processing Systems*, volume 37, 2024b. doi: 10.52202/079017-3609. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/cdf6f8e9fd9aeaf79b6024caec24f15b-Abstract-Datasets_and_Benchmarks_Track.html. Datasets and Benchmarks Track.

Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., and Han, S. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 38087–38099. PMLR, 2023. URL https://proceedings.mlr.press/v202/xiao23c.html.

Xiao, G., Tang, J., Zuo, J., Guo, J., Yang, S., Tang, H., Fu, Y., and Han, S. DuoAttention: Efficient long-context LLM inference with retrieval and streaming heads. *arXiv preprint arXiv:2410.10819*, 2024. URL https://arxiv.org/abs/2410.10819.

Yang, S., Lou, Y., Guo, J., Tang, J., Tang, H., and Han, S. TinyChat 2.0: Accelerating edge AI with efficient LLM and VLM deployment. MIT HAN Lab, 2024. URL https://hanlab.mit.edu/blog/tinychat20. Published December 12, 2024. Accessed September 30, 2026.

Yang, S., Guo, J., Tang, H., Hu, Q., Xiao, G., Tang, J., Lin, Y., Liu, Z., Lu, Y., and Han, S. LServe: Efficient long-sequence LLM serving with unified sparse attention. In *Proceedings of Machine Learning and Systems*, volume 7, 2025. URL https://arxiv.org/abs/2502.14866.

Ye, Z., Chen, L., Lai, R., Lin, W., Zhang, Y., Wang, S., Chen, T., Kasikci, B., Grover, V., Krishnamurthy, A.,

and Ceze, L. FlashInfer: Efficient and customizable attention engine for LLM inference serving. In *Proceedings of Machine Learning and Systems*, volume 7, 2025. URL https://proceedings.mlsys.org/paper_files/paper/2025/hash/dbf02b21d77409a2db30e56866a8ab3a-Abstract-Conference.html.

Yue, X., Ni, Y., Zhang, K., Zheng, T., Liu, R., Zhang, G., Stevens, S., Jiang, D., Ren, W., Sun, Y., Wei, C., Yu, B., Yuan, R., Sun, R., Yin, M., Zheng, B., Yang, Z., Liu, Y., Huang, W., Sun, H., Su, Y., and Chen, W. MMMU: A massive multi-discipline multimodal understanding and reasoning benchmark for expert AGI. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 9556–9567, 2024. URL https://openaccess.thecvf.com/content/CVPR2024/html/Yue_MMMU_A_Massive_Multi-discipline_Multimodal_Understanding_and_Reasoning_Benchmark_for_CVPR_2024_paper.html.

Zellers, R., Holtzman, A., Bisk, Y., Farhadi, A., and Choi, Y. HellaSwag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 4791–4800, 2019. doi: 10.18653/v1/P19-1472. URL https://aclanthology.org/P19-1472/.

AI USAGE DISCLOSURE

Generative AI tools, primarily OpenAI Codex and Claude Code, were used to assist with code generation, writing, and initial figure designs. For code generation, these tools primarily supported the development and iterative refinement of computational kernels. Verifiable objectives guided this process, enabling successive improvements in kernel efficiency through an approach similar to that explored in Kernel Design Agents (KDA) (NVlabs, 2026).

AI tools were also used to refine the manuscript's wording and improve the clarity and readability of explanations, helping communicate the work more effectively.